

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais
 - Processos órfãos e zumbis
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

TAREFA 8: criando uma chamada de kernel (de 13/03/2025 a 16/03/2025)

Esta tarefa foi projetada para guiar o aluno no procedimento para a confecção de uma chamada de kernel. Para auxiliar o aluno, esta prática será dividida em algumas etapas – estas etapas devem constar no relatório:

- (a) Adicione o protótipo de uma função de kernel chamada `do_mykernelcall()` no arquivo: `/usr/src/kernel/system.h`

```
...  
int do_mykernelcall(struct proc *caller, message *m_ptr);  
_PROTOTYPE( int do_mykernelcall, (struct proc * caller,  
message *m_ptr) );  
#endif /* SYSTEM_H */
```

- (b) Escreva a implementação de `do_mykernelcall()` em seu próprio arquivo fonte:

```
/usr/src/kernel/system/do_mykernelcall.c  
#include "kernel/system.h"  
#include <minix/endpoint.h>  
  
/*===== *  
* do_mykernelcall *
```

```

=====*/
PUBLIC int do_mykernelcall(struct proc *caller_ptr, message
*m_ptr)
{
    printf ("Esta e minha primeira chamada de kernel!");
    return(OK);
}

```

(c) Adicione do_mykernelcall ao Makefile para compilação:

```

/usr/src/kernel/system/Makefile.inc
# Makefile for system library implementation
.include <bsd.own.mk>

.PATH:  ${.CURDIR}/system
SRCS+=  \
        do_fork.c \
        do_exec.c \
        ...

        do_statectl.c \
        do_mykernelcall.c

```

(d) Mapeie SYS_MYKERNELCALL a do_mykernelcall() na tabela de chamadas de sistema: /usr/src/kernel/system.c

```

...
map(SYS_STATECTL, do_statectl); /* let a process control its
                                state */
map(SYS_MYKERNELCALL, do_mykernelcall); /* your kernel call
                                          */
...

```

(e) Adicione um protótipo para a função sys_mykernelcall no arquivo:

```

/usr/src/include/minix/syslib.h
...

```

```
int sys_mykernelcall(unsigned flags, endpoint_t proc_ep);
_PROTOTYPE( int sys_mykernelcall, (unsigned flags, endpoint_t
proc_ep));

...
```

- (f) Adicione o número de chamada para sys_mykernelcall ao vetor de chamada e incremente sua dimensão: /usr/src/include/minix/com.h

```
...
#define SYS_STATECTL (KERNEL_CALL + 55) /* sys_statectl() */
#define SYS_MYKERNELCALL (KERNEL_CALL + 56) /*
sys_mykernelcall() */

/* Total */
#define NR_SYS_CALLS 57 /* number of kernel calls */
...
```

- (g) Adicione o serviço MYKERNELCALL à tabela de serviços do sistema: /usr/src/commands/service/service.c;
construa a tabela de serviços do sistema e instale-a com make; make install a partir de /usr/src/commands/service/

```
...
struct
{
    char *label;
    int call_nr;
} system_tab[]=
{
    ...
    { "MYKERNELCALL",          SYS_MYKERNELCALL },
    { NULL,                    0 }
};
...
```

- (h) Escreva sua implementação da função sys_mykernelcall em um novo arquivo:

```

/usr/src/lib/libsys/sys_mykernelcall.c
#include "syslib.h"

PUBLIC int sys_mykernelcall(unsigned flags, endpoint_t
proc_ep)
{
    message m;

    return(_kernel_call(SYS_MYKERNELCALL, &m));
}

```

- (i) Adicione `sys_mykernelcall.c` ao `Makefile`, e instale-a em `/usr/lib/${MACHINE_ARCH}` com `make install`.

```

# Makefile for libsys

LIB=          sys
SRCS=  \
...
    sys_mykernelcall.c \
...

```

- (j) Reconstrua o kernel e instale-o novamente com `make`; `make install` a partir de `/usr/src/tools`.
- (k) Teste a nova chamada de kernel usando a chamada de sistema da tarefa anterior. Adicione a seguinte linha à implementação da chamada de sistema `do_mycall()`
- ```

_kernel_call (SYS_MYKERNELCALL, &m);

```